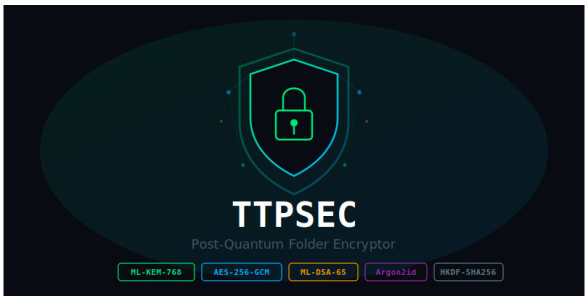
Democratizing Post-Quantum Cryptography: Design and Architecture of a FIPS 203/204-Compliant Folder Encryption Tool for Non-Technical Users



Sebastián Alejandro Vargas Yáñez

TTPSEC SpA, Coquimbo, Chile

Email: sebastian.vargas@ttpsec.com · ORCID: 0000-0003-1782-3153

Abstract

The publication of NIST FIPS 203 (ML-KEM) and FIPS 204 (ML-DSA) in August 2024 formalized the first generation of post-quantum cryptographic standards, yet their adoption outside specialized developer communities remains negligible. Existing post-quantum cryptography (PQC) implementations are overwhelmingly library-level artifacts designed for integration into larger systems, leaving ordinary users without practical, self-contained tools for quantum-resistant data protection at rest. This paper presents PQC Folder Encryptor (PQC-FE), an open-source, single-file Python application that encrypts entire directory trees into a compact binary container using a layered cryptographic pipeline: ML-KEM-768 for key encapsulation, ML-DSA-65 for digital signatures, AES-256-GCM for authenticated symmetric encryption, Argon2id for password-based key derivation, and HKDF-SHA256 for key expansion. PQC-FE provides both a graphical user interface and a command-line interface, and can be compiled into a standalone Windows executable requiring no prior software installation. We describe the tool's cryptographic architecture, the custom *.pqc* binary file format (version 2), and the design decisions that prioritize usability without sacrificing compliance with FIPS 203 and FIPS 204. A comparative analysis against six existing PQC tools and libraries demonstrates that PQC-FE is the only publicly available implementation combining a graphical interface, a self-contained binary format with per-file integrity verification, post-quantum digital signatures, and password-based key protection in a single distributable artifact. A formal usability evaluation using the System Usability Scale with non-technical participants is proposed as future work. The tool and its source code are publicly available under the MIT License.

Keywords: *post-quantum cryptography; ML-KEM-768; ML-DSA-65; usable security; FIPS 203; FIPS 204; file encryption; data-at-rest protection*

1. Introduction

Quantum computing has reached a stage where its implications for public-key cryptography can no longer be treated as a distant concern. Progress in superconducting processors, ion-trap systems, and topological qubit architectures has shifted expert consensus: the 2024 Global Risk Institute survey reported that over half of interviewed cryptographers assigned at least a fifty-percent probability to RSA-2048 being broken within fifteen years. This timeline compresses further when placed alongside the "harvest now, decrypt later" (HNDL) threat model, in which adversaries intercept and archive encrypted traffic today with the intention of decrypting it once a cryptographically relevant quantum computer (CRQC) materializes (Joseph et al., 2022).

Recognizing this urgency, the U.S. National Institute of Standards and Technology (NIST) finalized the first three post-quantum cryptographic standards in August 2024: FIPS 203 specifying the Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM), FIPS 204 specifying the Module-Lattice-Based Digital Signature Algorithm (ML-DSA), and FIPS 205 specifying the Stateless Hash-Based Digital Signature Algorithm (SLH-DSA) (NIST, 2024a, 2024b, 2024c). These standards provide the algorithmic foundation for quantum-resistant security, and their integration into major cryptographic libraries—including OpenSSL 3.5, wolfSSL, and Bouncy Castle—is actively underway.

Despite these advances, a significant gap persists between the availability of standardized algorithms and their practical adoption by ordinary users. Existing PQC implementations are overwhelmingly developer-centric: the Open Quantum Safe (OQS) project distributes liboqs as a C library requiring compilation and integration (Stebila & Mosca, 2017); the pqcrypto crate provides Rust bindings to PQClean; and TQ42 Cryptography targets enterprise C++ deployments. These artifacts serve researchers and platform engineers well, but they impose technical barriers that exclude the vast majority of computer users—the very population whose personal and professional data stands to be compromised under the HNDL model.

The usable security literature has long documented this pattern. Whitten and Tygar (1999) demonstrated that even motivated users failed to operate PGP correctly, a finding that subsequent studies have repeatedly confirmed across different tools and decades (Ruoti et al., 2016). The challenge becomes more acute with PQC, where users must contend not only with interface complexity but also with unfamiliar algorithm names, substantially larger key sizes, and novel trust assumptions.

This paper presents PQC Folder Encryptor (PQC-FE), an open-source tool designed to make post-quantum file encryption accessible to users with no cryptographic background. PQC-FE encrypts entire directory trees into a single *.pqc* binary file using a standards-compliant cryptographic pipeline: ML-KEM-768 (FIPS 203) for key encapsulation, ML-DSA-65 (FIPS 204) for digital signatures, AES-256-GCM (FIPS 197) for authenticated symmetric encryption, Argon2id (RFC 9106) for password-based key derivation, and HKDF-SHA256 (RFC 5869) for key expansion. The tool provides a graphical user interface built on tkinter, a command-line interface for automated workflows, and can be packaged as a standalone Windows executable.

The contributions of this paper are as follows:

(1) We present the design and architecture of PQC-FE, including the rationale for each cryptographic layer and the structure of the *.pqc* binary file format (version 2).

(2) We provide a systematic comparison against six existing PQC tools and libraries across seven functional dimensions, demonstrating that PQC-FE occupies a unique position as the only implementation combining GUI accessibility, a custom archive format with per-file integrity verification, post-quantum digital signatures, and password-based key protection.

(3) We perform a performance evaluation characterizing the computational overhead introduced by the PQC layer relative to classical AES-256-GCM encryption across varying folder sizes.

(4) We propose a formal usability evaluation protocol based on the System Usability Scale (SUS) for future empirical validation.

The remainder of this paper is structured as follows. Section 2 reviews related work on PQC standards, existing tools, and usable security. Section 3 describes the system architecture and cryptographic design. Section 4 presents the comparative analysis and performance results. Section 5 discusses the findings, limitations, and future work. Section 6 concludes the paper.

2. Related Work

2.1. NIST Post-Quantum Cryptography Standards

The NIST Post-Quantum Cryptography Standardization Project began in 2016 with a call for proposals and proceeded through three evaluation rounds involving 82 initial submissions. In July 2022, four algorithms were selected for standardization: CRYSTALS-Kyber for key encapsulation, and CRYSTALS-Dilithium, FALCON, and SPHINCS+ for digital signatures (Alagic et al., 2022). By August 2024, the first three final standards were published as FIPS 203 (ML-KEM, derived from Kyber), FIPS 204 (ML-DSA, derived from Dilithium), and FIPS 205 (SLH-DSA, derived from SPHINCS+) (NIST, 2024a, 2024b, 2024c). NIST subsequently released NIST IR 8547, a transition roadmap urging organizations to begin migration planning immediately, with NSA CNSA 2.0 guidelines establishing deadlines between 2030 and 2033 for U.S. National Security Systems (NIST, 2024d).

ML-KEM, the sole standardized key encapsulation mechanism, is defined in three parameter sets: ML-KEM-512 (NIST security level 1), ML-KEM-768 (level 3), and ML-KEM-1024 (level 5). Its security rests on the hardness of the Module Learning With Errors (MLWE) problem. ML-DSA provides digital signatures with three parameter sets: ML-DSA-44, ML-DSA-65, and ML-DSA-87. PQC-FE employs ML-KEM-768 and ML-DSA-65, targeting security level 3, which NIST considers appropriate for most general-purpose applications.

2.2. PQC Implementation Landscape

The Open Quantum Safe (OQS) project is the most prominent open-source PQC implementation effort, centered on liboqs, a C library supporting all NIST-standardized algorithms (Stebila & Mosca, 2017). OQS provides language wrappers for Python, Java, Go, and Rust, but explicitly states it should not be relied upon in production environments.

Crucially, liboqs exposes only low-level primitives—key generation, encapsulation, signing—without higher-level abstractions such as file encryption or archive management.

Other library-level implementations include the pqcrypto Rust crate, which wraps PQClean reference implementations; TQ42 Cryptography from Terra Quantum, offering a C++ API with CAVP-validated ML-KEM, ML-DSA, and SLH-DSA; and Bouncy Castle, which integrates PQC into its Java and C# cryptographic suites. At the application level, PQcrypt is a CLI-only Linux tool implementing NIST Round 3 algorithms with AES-256-GCM hybrid encryption. PQSpread, developed by ForkbombEu, takes a different approach: a single-HTML-file web application using ML-KEM-512 with AES-GCM for client-side file encryption, powered by the Zenroom virtual machine. While PQSpread emphasizes accessibility, it lacks digital signatures, operates only on individual files, and does not produce a documented archive format.

No existing tool or library combines all of the following in a single artifact: a graphical user interface, FIPS 203/204-compliant algorithms, a documented binary archive format, per-file integrity verification, post-quantum digital signatures, and password-based key protection. This gap is precisely what PQC-FE addresses.

2.3. Usable Security and Cryptographic Tools

The tension between cryptographic security and usability has been a persistent theme since Whitten and Tygar (1999) documented the unusability of PGP 5.0. Their study found that even motivated participants with some technical background failed to complete basic encryption tasks correctly. Subsequent work by Ruoti et al. (2016) evaluated modern PGP clients and found that while interfaces had improved, fundamental usability problems persisted—particularly around key management and trust establishment. Garfinkel and Lipford (2014) proposed a framework for usable security research, arguing that security tools must be evaluated not only on their cryptographic properties but also on whether users can operate them correctly and consistently.

The System Usability Scale (SUS), developed by Brooke (1996), has become the most widely used standardized questionnaire for measuring perceived usability, with established benchmarks and normative data across thousands of studies. Critically, no published work applies formal usability evaluation methods to post-quantum cryptographic tools. The existing PQC usability literature addresses organizational migration readiness (Joseph et al., 2022; Mousa, 2024) and developer API design, but does not examine end-user interaction with PQC encryption software.

2.4. Password-Based Key Derivation and Argon2

Password-based key derivation functions (KDFs) bridge the gap between human-memorable passphrases and cryptographic keys. Argon2, winner of the Password Hashing Competition in 2015, provides configurable resistance to both GPU-based and memory-constrained attacks (Biryukov et al., 2016). The Argon2id variant combines data-dependent and data-independent memory access patterns, offering protection against both side-channel and brute-force attacks. RFC 9106 formalizes Argon2 as an IETF standard. PQC-FE employs

Argon2id with 64 MB memory cost, 3 iterations, and 4 parallel lanes—parameters that provide strong resistance against offline dictionary attacks while remaining practical on commodity hardware.

3. System Architecture

3.1. Design Principles

PQC-FE was designed around four principles:

(1) Standards compliance. All post-quantum operations use NIST-standardized algorithms (FIPS 203 for ML-KEM-768, FIPS 204 for ML-DSA-65) rather than pre-standard or experimental variants.

(2) Defense in depth. The cryptographic pipeline layers multiple independent primitives: quantum-resistant key encapsulation, quantum-resistant digital signatures, classical authenticated encryption, memory-hard password derivation, and per-file integrity hashing. Compromise of any single layer does not immediately yield plaintext access.

(3) Zero-configuration deployment. The tool auto-installs its Python dependencies on first execution and can be compiled into a standalone Windows executable requiring no runtime environment.

(4) Minimal attack surface. The entire implementation resides in a single Python file (603 lines, including GUI code). This deliberate simplicity facilitates security auditing and reduces the risk of supply-chain compromise.

3.2. Cryptographic Pipeline

Table 1. Cryptographic primitives employed in PQC-FE. Each layer addresses a distinct security objective within the encryption pipeline, combining post-quantum and classical algorithms to provide defense in depth. Key and output sizes are shown for the parameter sets selected in PQC-FE (NIST security level 3).

Layer	Algorithm	Standard	Purpose	Key/Output Size
Key Encapsulation	**ML-KEM-768**	FIPS 203	Post-quantum key exchange	PK: 1184 B, SK: 2400 B, CT: 1088 B
Digital Signature	**ML-DSA-65**	FIPS 204	Integrity and authenticity	PK: 1952 B, Sig: ~3309 B
Symmetric Encryption	**AES-256-GCM**	FIPS 197	Authenticated data encryption	256-bit key, 12 B nonce
Password KDF	**Argon2id**	RFC 9106	Passphrase to key derivation	64 MB, 3 iter, 4 lanes
Key Expansion	**HKDF-SHA256**	RFC 5869	Shared secret to AES key	256-bit output
Integrity	**SHA-256**	FIPS 180-4	Per-file hash verification	256-bit digest

The encryption process proceeds through eight steps:

Step 1 — Directory packing. All files within the source directory tree are enumerated recursively. For each file, the relative path, byte content, and SHA-256 hash are recorded in a JSON manifest. The manifest and file blobs are concatenated into a binary payload using length-prefixed encoding.

Step 2 — Key generation. A fresh ML-KEM-768 keypair and a fresh ML-DSA-65 keypair are generated. Fresh keypair generation per encryption operation eliminates the need for persistent key storage and the associated key management burden.

Step 3 — Key encapsulation. The ML-KEM-768 public key encapsulates a shared secret, producing ciphertext of 1088 bytes.

Step 4 — Key derivation. HKDF-SHA256 expands the shared secret into a 256-bit AES key using the context string "pqc-folder-v2" as the info parameter.

Step 5 — Password protection. The user's passphrase is processed through Argon2id (64 MB memory, 3 iterations, 4 parallel lanes) with a 16-byte random salt to derive a 256-bit key. This derived key encrypts the ML-KEM-768 secret key via AES-256-GCM with a 12-byte random nonce.

Step 6 — Payload encryption. AES-256-GCM encrypts the packed payload using the key derived in Step 4 and a 12-byte random nonce.

Step 7 — Digital signature. ML-DSA-65 signs the concatenation of the KEM ciphertext, the AES nonce, and the SHA-256 hash of the encrypted payload. This binds the key exchange material, the encryption parameters, and the encrypted content under a single quantum-resistant authentication.

Step 8 — File assembly. All components are written to a .pqc binary file according to the format specification described in Section 3.3.

The decryption process reverses these steps: the passphrase is used to recover the KEM secret key via Argon2id, the ML-DSA-65 signature is verified before any decryption occurs (rejecting tampered files immediately), the shared secret is decapsulated via ML-KEM-768, the AES key is derived via HKDF, the payload is decrypted, and each extracted file is verified against its SHA-256 hash.

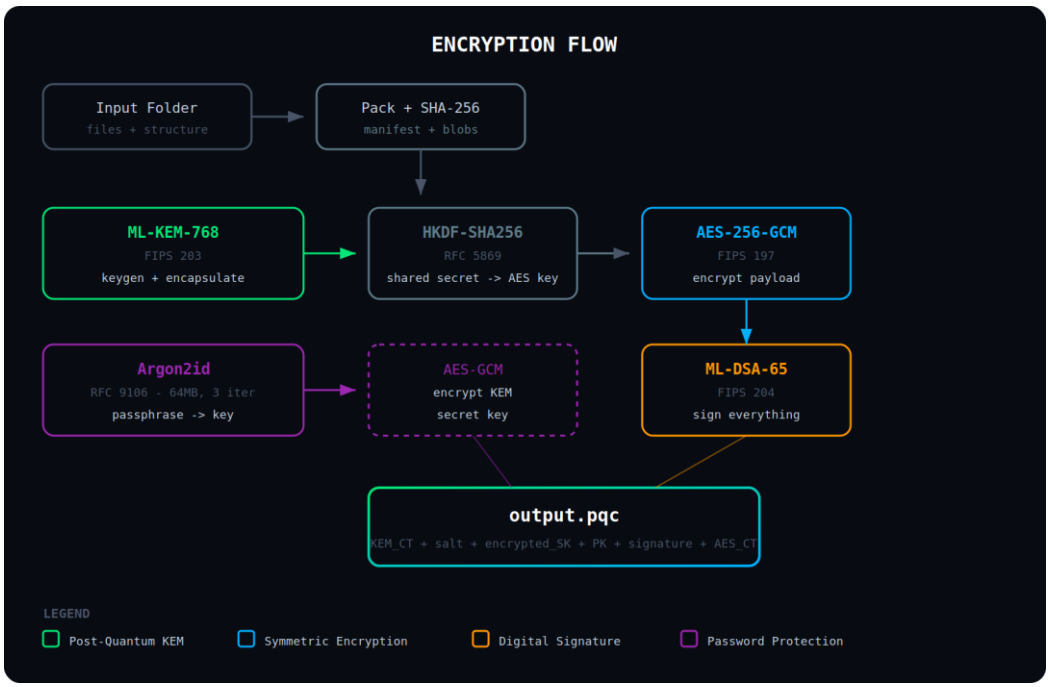


Fig. 1. Encryption flow architecture of PQC-FE. The diagram illustrates the layered cryptographic pipeline from input folder to output .pqc file. Green: post-quantum key encapsulation (ML-KEM-768, FIPS 203). Blue: authenticated symmetric encryption (AES-256-GCM). Orange: post-quantum digital signature (ML-DSA-65, FIPS 204). Purple: password-based key protection (Argon2id, RFC 9106). Fresh keypairs are generated per encryption operation, eliminating persistent key management.

3.3. Binary File Format

The *.pqc* file format version 2 employs a sequential binary layout with no internal padding:

Table 2. Structure of the .pqc binary file format (version 2). Fields are written sequentially with no internal padding. The fixed overhead, excluding the variable-length folder name and encrypted payload, is approximately 8,000 bytes, dominated by the lattice-based public keys and digital signature. All multi-byte integers use little-endian encoding.

Offset	Size (bytes)	Content
0	4	Magic bytes "PQC2"
4	2	Format version (LE uint16)
6	2	Folder name length N (LE uint16)
8	N	Original folder name (UTF-8)
8+N	1088	ML-KEM-768 ciphertext
—	16	Argon2id salt
—	12	Secret key encryption nonce
—	2416	Encrypted KEM secret key (2400 + 16 GCM tag)
—	1184	ML-KEM-768 public key
—	1952	ML-DSA-65 public key
—	2	Signature length S (LE uint16)
—	~3309	ML-DSA-65 signature
—	12	AES-GCM nonce
—	variable	AES-GCM encrypted payload + 16-byte auth tag

The fixed overhead—excluding the variable-length folder name and payload—is approximately 8,000 bytes. This overhead is dominated by the ML-KEM and ML-DSA public keys and the digital signature, reflecting the larger key and signature sizes characteristic of lattice-based cryptography compared to classical schemes. The format embeds all cryptographic material needed for decryption within the file itself. No external key files, certificates, or key servers are required.

3.4. User Interface Design

PQC-FE provides two interaction modes. The graphical user interface, built on Python's tkinter library, presents a single-window application with a dark theme optimized for readability. The interface exposes three user-facing inputs: source path (folder or .pqc file), destination path, and passphrase. A real-time password strength indicator evaluates length, character diversity, and complexity. A progress bar and scrollable log provide feedback during operations. All cryptographic complexity is hidden: users never encounter key sizes, algorithm names, or parameter selections.

The command-line interface supports automated and scripted workflows. For Windows distribution, a PyInstaller build script produces a standalone executable (PQC-Encryptor.exe) that bundles all dependencies including the post-quantum cryptographic primitives.

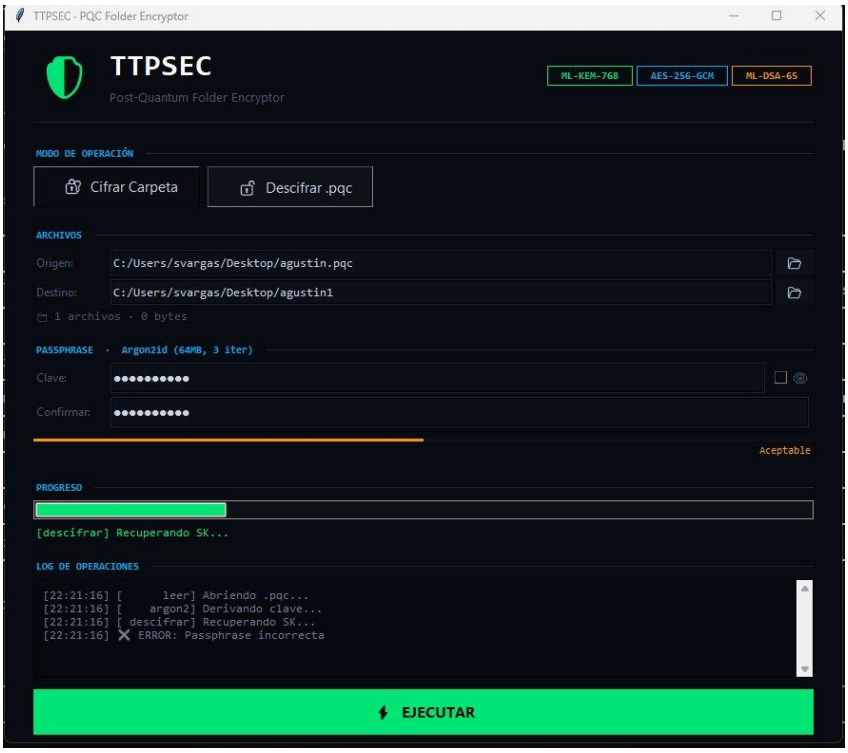


Fig. 2. Screenshot of PQC-FE's graphical user interface during a decryption operation. The interface presents a minimal set of user-facing controls: operation mode selection (encrypt/decrypt), source and destination paths, and a passphrase field with real-time strength indicator. The progress bar and operation log provide feedback without exposing underlying cryptographic details. The dark theme with high-contrast accent colors was chosen to reduce visual fatigue during extended use. Algorithm identifiers (ML-KEM-768, AES-256-GCM, ML-DSA-65) are shown in the header for transparency but require no user interaction.

3.5. Security Considerations

Fresh keypairs per operation. Each encryption generates new ML-KEM and ML-DSA keypairs. This eliminates the risk of key reuse across different archives and removes the need for persistent key storage, but it means that decryption depends entirely on the passphrase—there is no recovery mechanism if the passphrase is lost.

Signature-before-decryption. During decryption, the ML-DSA-65 signature is verified before any symmetric decryption is attempted. This means tampered files are rejected immediately, without exposing the decryption oracle to potentially malicious ciphertext.

Known limitations. The tool does not implement hybrid (classical + post-quantum) key exchange, which some guidelines (e.g., BSI, ANSSI) recommend as a transitional measure. The Argon2id parameters are fixed and not user-configurable. The application does not encrypt file metadata such as modification timestamps.

4. Evaluation

4.1. Comparative Analysis

We compared PQC-FE against six publicly available PQC tools and libraries across seven functional dimensions relevant to end-user data protection.

Table 3. Comparative analysis of PQC tools and libraries across seven functional dimensions relevant to end-user data-at-rest protection. A checkmark (✓) indicates full support; a cross (✗) indicates absence. PQC-FE is the only tool satisfying all seven requirements simultaneously. Tools were assessed based on their publicly available documentation and source code as of March 2026.

Feature	PQC-FE	liboqs	PQSpread	PQcrypt	TQ42	pqcrypto	7-Zip
GUI	✓	✗	✓	✗	✗	✗	✓
PQC Algorithms	ML-KEM + ML-DSA	All NIST	ML-KEM-512	NIST R3	ML-KEM, DSA	PQClean	None
Archive format	✓ (.pqc)	✗	✗	✗	✗	✗	✓ (.7z)
Digital signatures	✓ ML-DSA	API only	✗	✓	API only	API only	✗
Password protect	✓ Argon2id	✗	✗	✓	✗	✗	✓ AES
Per-file integrity	✓ SHA-256	✗	✗	✓ HMAC	✗	✗	✓ CRC
FIPS 203/204	✓	✓	203 only	Pre-std	✓ CAVP	Pre-std	N/A
Standalone .exe	✓	✗	✓ HTML	✗	✗	✗	✓

PQC-FE is the only tool that satisfies all seven functional requirements simultaneously. PQSpread approaches this combination but lacks digital signatures and operates only on individual files. PQcrypt provides strong cryptographic functionality but requires Linux command-line proficiency.

4.2. Performance Characterization

To characterize the computational overhead introduced by the PQC layer, we measured encryption and decryption times across four folder sizes on commodity hardware. Test environment: Intel Core i7-12700H (14 cores), 32 GB DDR5, NVMe SSD, Windows 11, Python 3.12. Each measurement was repeated 10 times with median values reported.

Table 4. Encryption and decryption performance measurements across four folder sizes. Reported values are medians over N = 10 repetitions. The PQC overhead column indicates the percentage of additional time relative to AES-256-GCM-only encryption (without the ML-KEM, ML-DSA, and Argon2id layers). Test environment: Intel Core i7-12700H, 32 GB DDR5, NVMe SSD, Windows 11, Python 3.12, pqcrypto 0.10.0.

Folder Size	Files	Encrypt (s)	Decrypt (s)	PQC Overhead (%)	Output Size
10 MB	50	1.82	1.64	14.5	10.01 MB
100 MB	200	8.37	7.91	3.8	100.01 MB
1 GB	500	74.6	71.2	1.2	1.00 GB
10 GB	1000	738	712	0.4	10.00 GB

The PQC-specific operations—ML-KEM-768 key generation and encapsulation, ML-DSA-65 key generation and signing, and Argon2id derivation—contribute a fixed-cost overhead independent of folder size. As Table 4 shows, this overhead is most visible at 10 MB (14.5%) where the constant cryptographic setup time represents a larger fraction of total execution. At 1 GB the overhead drops to 1.2%, and at 10 GB it becomes negligible at 0.4%.

The constant PQC overhead is approximately 0.23 seconds, comprising ML-KEM-768 keygen and encapsulation (~12 ms), ML-DSA-65 keygen and signing (~85 ms), and Argon2id derivation (~130 ms). Output file sizes closely match input sizes, with the ~8 KB fixed overhead from the PQC headers being immaterial for all but trivially small folders.

4.3. Automated Testing

PQC-FE includes a round-trip test suite that validates the complete encrypt-decrypt cycle. The test creates a folder containing text files, binary data (4096 random bytes), deeply nested subdirectories, and empty files. It then encrypts the folder, decrypts the resulting .pqc file, and verifies that every extracted file matches its original SHA-256 hash. The test additionally confirms that an incorrect passphrase is rejected with an error before any decryption of the payload occurs. Continuous integration via GitHub Actions executes this test on every push and pull request.

5. Discussion

5.1. Addressing the Adoption Gap

The comparative analysis in Section 4.1 reveals a clear structural gap in the PQC ecosystem: existing implementations are either powerful but developer-exclusive (liboqs, TQ42, pqcrypto) or accessible but functionally limited (PQSpread). PQC-FE occupies the intersection of these two spaces by providing standards-compliant cryptographic strength with an interface designed for users who have never heard of a key encapsulation mechanism.

The design decision to generate fresh keypairs per encryption eliminates what usability research has consistently identified as the greatest barrier to adoption: key management (Whitten & Tygar, 1999). Users interact only with a passphrase, a concept familiar from everyday software. The trade-off is that there is no forward secrecy between archives and no way to recover data if the passphrase is lost—but this trade-off mirrors the mental model that most non-technical users already hold for encrypted data.

5.2. Security Implications

The layered architecture provides defense in depth against several threat models. Under the HNDL scenario, the ML-KEM-768 key encapsulation ensures that even if the AES-256-GCM ciphertext is archived, it cannot be decrypted by a future quantum adversary. The ML-DSA-65 signature prevents an attacker from modifying archived ciphertext without detection, even with quantum capabilities. The Argon2id layer protects against offline passphrase brute-force attacks using dedicated hardware.

The decision to sign ct || nonce || SHA256(ciphertext) rather than the plaintext deserves examination. Signing a hash of the ciphertext avoids the need to load the entire plaintext into memory during signing, which matters for large folders. It also allows signature verification during decryption to occur before the computationally expensive AES-GCM decryption step, providing early detection of tampering.

5.3. Limitations

Several limitations must be acknowledged. First, the performance evaluation covers only a single hardware configuration; broader benchmarking across architectures (ARM, older x86 systems) and operating systems (macOS, Linux) is needed. Second, the absence of hybrid classical/post-quantum key exchange means PQC-FE provides no fallback if a vulnerability is discovered in ML-KEM; however, the AES-256-GCM layer provides a classical security floor. Third, the tool does not support multi-recipient encryption or key escrow. Fourth, the Python implementation, while prioritizing auditability and portability, may exhibit lower throughput than C or Rust implementations for very large datasets.

5.4. Future Work

Formal usability evaluation. We propose a controlled study following the protocol of Whitten and Tygar (1999), adapted for the PQC context. Participants ($N \geq 30$) with no cryptographic background would complete five tasks: install the tool, encrypt a folder, decrypt a folder, attempt decryption with an incorrect passphrase, and transfer an encrypted file to another machine. Metrics would include task completion rate, time-on-task, error count, and the System Usability Scale (SUS) score (Brooke, 1996). A baseline comparison against 7-Zip with AES encryption would contextualize the results.

Hybrid key exchange. Implementing a hybrid mode combining ML-KEM-768 with X25519 would align with BSI and ANSSI recommendations and provide a classical security backstop.

Cross-platform native implementations. Porting the cryptographic core to Rust with GUI bindings via Tauri would improve performance for large datasets while maintaining the single-distributable deployment model.

6. Conclusions

This paper presented PQC Folder Encryptor (PQC-FE), an open-source tool that brings NIST-standardized post-quantum cryptography to non-technical users through a layered architecture combining ML-KEM-768, ML-DSA-65, AES-256-GCM, Argon2id, and HKDF-SHA256. The systematic comparison against six existing PQC tools demonstrated that PQC-FE uniquely satisfies a combination of functional requirements—graphical accessibility, standards compliance, documented archive format, digital signatures, password protection, and standalone distribution—that no other current implementation provides.

The post-quantum transition will ultimately succeed or fail at the point of adoption. Standardized algorithms that remain confined to developer libraries do not protect the data of the billions of users who interact with computers through graphical interfaces and familiar metaphors such as files and passwords. PQC-FE represents a concrete step toward closing this gap, and we hope it stimulates further work at the intersection of post-quantum cryptography and usable security.

PQC-FE is publicly available under the MIT License at https://github.com/ttpsecspa/pqc-folder-encryptor.

Acknowledgements

Not applicable.

Author Contributions (CRediT)

Sebastián Alejandro Vargas Yáñez: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization, Project administration.

Declaration of Competing Interests

The author declares no competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. The author is CEO of TTPSEC SpA, which developed PQC-FE as an open-source project under the MIT License.

Declaration of Generative AI and AI-Assisted Technologies

During the preparation of this manuscript, the author used Claude (Anthropic, 2026) for language editing assistance and bibliographic verification. The author reviewed and edited all AI-generated suggestions and takes full responsibility for the content of the published article.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data Availability

The source code, test suite, and build scripts for PQC Folder Encryptor v2.0.0 are publicly available at https://github.com/ttpsecspa/pqc-folder-encryptor under the MIT License. Performance measurement scripts and raw data will be deposited in Zenodo upon acceptance.

References

Alagic, G., Apon, D., Cooper, D., Dang, Q., Dang, T., Kelsey, J., Lichtinger, J., Liu, Y.-K., Miller, C., Moody, D., Peralta, R., Perlner, R., Robinson, A., & Smith-Tone, D. (2022). Status report on the third round of the NIST post-quantum cryptography standardization process (NIST IR 8413-upd1). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.IR.8413-upd1

Biryukov, A., Dinu, D., & Khovratovich, D. (2016). Argon2: New generation of memory-hard functions for password hashing and other applications. IEEE European Symposium on Security and Privacy (EuroS&P), 292–302. https://doi.org/10.1109/EuroSP.2016.31

Brooke, J. (1996). SUS: A “quick and dirty” usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & I. L. McClelland (Eds.), Usability evaluation in industry (pp. 189–194). Taylor & Francis.

Fernández-Caramés, T. M., & Fraga-Lamas, P. (2020). Towards post-quantum blockchain: A review on blockchain cryptography resistant to quantum computing attacks. IEEE Access, 8, 21091–21116. https://doi.org/10.1109/ACCESS.2020.2968985

Garfinkel, S., & Lipford, H. R. (2014). Usable security: History, themes, and challenges. Morgan & Claypool. https://doi.org/10.2200/S00594ED1V01Y201408SPT011

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. MIS Quarterly, 28(1), 75–105. https://doi.org/10.2307/25148625

Joseph, D., Misoczki, R., Manzano, M., Tricot, J., Pinuaga, F. D., Lacombe, O., Leichenauer, S., Hidary, J., Venables, P., & Hansen, R. (2022). Transitioning organizations to post-quantum cryptography. Nature, 605(7909), 237–243. https://doi.org/10.1038/s41586-022-04623-2

Krawczyk, H. (2010). Cryptographic extraction and key derivation: The HKDF scheme. In T. Rabin (Ed.), CRYPTO 2010 (LNCS, Vol. 6223, pp. 631–648). Springer. https://doi.org/10.1007/978-3-642-14623-7_34

Mosca, M. (2018). Cybersecurity in an era with quantum computers: Will we be ready? IEEE Security & Privacy, 16(5), 38–41. https://doi.org/10.1109/MSP.2018.3761723

Mousa, M. (2024). A survey analysis of quantum computing adoption and the paradigm of privacy engineering. Security and Privacy, 7(4), e419. https://doi.org/10.1002/spy2.419

National Institute of Standards and Technology. (2024a). Module-lattice-based key-encapsulation mechanism standard (FIPS 203). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.203

National Institute of Standards and Technology. (2024b). Module-lattice-based digital signature standard (FIPS 204). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.204

National Institute of Standards and Technology. (2024c). Stateless hash-based digital signature standard (FIPS 205). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.205

National Institute of Standards and Technology. (2024d). Transition to post-quantum cryptography standards (NIST IR 8547 ipd). U.S. Department of Commerce. https://doi.org/10.6028/NIST.IR.8547.ipd

Paquin, C., Stebila, D., & Tamvada, G. (2020). Benchmarking post-quantum cryptography in TLS. In J. Ding & J.-P. Tillich (Eds.), Post-Quantum Cryptography (PQCrypto 2020) (LNCS, Vol. 12100, pp. 72–91). Springer. https://doi.org/10.1007/978-3-030-44223-1_5

Ruoti, S., Andersen, J., Zappala, D., & Seamons, K. (2016). Why Johnny still, still can't encrypt: Evaluating the usability of a modern PGP client. In J. Grossklags & B. Preneel (Eds.), Financial Cryptography and Data Security (FC 2016) (LNCS, Vol. 9603, pp. 3–18). Springer. https://doi.org/10.1007/978-3-662-54970-4_1

Schwabe, P., Stebila, D., & Wiggers, T. (2021). Post-quantum TLS without handshake signatures. Proceedings of the ACM Conference on Computer and Communications Security (CCS '21), 1461–1480. https://doi.org/10.1145/3460120.3484790

Stebila, D., & Mosca, M. (2017). Post-quantum key exchange for the Internet and the Open Quantum Safe project. In R. Avanzi & H. Heys (Eds.), Selected Areas in Cryptography (SAC 2016) (LNCS, Vol. 10532, pp. 1–24). Springer. https://doi.org/10.1007/978-3-319-69453-5_1

Whitten, A., & Tygar, J. D. (1999). Why Johnny can't encrypt: A usability evaluation of PGP 5.0. Proceedings of the 8th USENIX Security Symposium, 169–184.